

ENGINEERING SUBMISSION · CONNECTED VEHICLE INTELLIGENCE HACKATHON

VoltSight

EV Fleet Range-Risk & Charge Orchestration

A streaming platform that tells every EV in a 100,000-vehicle fleet whether it can still reach a charger, why, and where to charge — within seconds of the telemetry that changes the answer.

98.3K

events/s measured
(100K vehicles, 60 s)

0 lost

3× burst (40 s) and
5 chaos scenarios

26.8 / 41.0 ms

API p95 / p99
(targets 200 / 500 ms)

58 / 0 / 3

ZAP baseline: pass /
fail / warning groups

Repository

github.com/SohamB4746Y/
voltsight-connected-vehicle-intelligence

Requirement matrix

60 requirements: PASS 28 · PARTIAL 28 ·
BLOCKED 2 · FAIL 2



repository

All data is synthetic. Every number in this document is traced to a file in the repository; items not verified are labelled NOT VERIFIED.
Motorq is named only as an industry reference in the problem statement; this project has no affiliation with Motorq.

00

Why VoltSight deserves attention

Seven proof points, each tied to a file in the repository. Targets and measurements are kept apart throughout.

#	Claim	Evidence (measured unless stated)	Where
1	100K-vehicle scale, end to end	100,000 vehicles streamed in real time through the mTLS gateway: 98.3K events/s for 60 s, 6.0M events, 0 failed; ClickHouse distinct events equal the worker's. A silent data-loss bug was found by this run and fixed.	evidence/G4
2	Real-time range-risk decision	Multi-source Dijkstra overlay answers "distance to the nearest <i>available</i> charger" in 3.6 ns per event; an outage updates it incrementally in 70.7 μs. EWMA estimator detected 25/25 simulated strandings with a 53 min median lead.	internal/rangerisk, evidence/G6
3	Resilience with proof	Kill/restart of worker, sink, Redis, ClickHouse, Kafka under load: 0 events lost in all five; the method found and fixed two real defects.	evidence/G13
4	ML that beats strong baselines	GBM MAE 0.01736 vs 0.0274 kWh/km (per-vehicle history) on unseen vehicles; CI of the difference excludes 0; leakage checks pass. Labels are simulated.	evidence/G8
5	Agentic Copilot with guardrails in code	8 typed tools, tenant from the JWT, step/tool caps, citations, two-phase human-approved writes, every step audited. Default provider is a deterministic stub.	internal/copilot, evidence/G10
6	Security and tenant isolation	OIDC + PKCE, RBAC, PostgreSQL RLS (forced), mTLS 1.3 for devices; 31/31 scripted checks as real users; ZAP baseline 58 pass / 0 fail / 3 warning groups.	evidence/live-deployment, evidence/security
7	Honest engineering	Critical-alert target (< 5 s) is not met at 100K events/s on one host (p50 9.8 s, p99 17.0 s). Reported, explained and in the matrix as FAIL.	evidence/G14

2-minute judge path

Page	Read	Gets you
2	Why VoltSight deserves attention	WHAT and WHY
4	Solution and workflow	HOW
5	System architecture (Figure 1)	TECHNOLOGY
6	Live dashboard, map, risk → charger decision (Figures 5–8)	PROOF it runs
11	Agentic Copilot (Figure 12)	AI with guardrails
15	Performance: target vs measured	VALUE and honesty

What was verified, and what was not

VERIFIED (measured) 100K-vehicle run at 98.3K ev/s for 60 s · 3× burst 40 s, 0 lost · 5 chaos scenarios, 0 lost · API p95/p99 26.8/41.0 ms · 31/31 live security checks · ML vs baselines · ZAP baseline 58/0/3	PARTIAL Real basemap tiles (markers-only fallback verified) · DAST (unauthenticated) · Helm (API-server acceptance only) · Terraform (validate only) · contract tests (Protobuf, no Pact)	NOT MET / NOT RUN Critical alert < 5 s at 100K ev/s (p99 17.0 s) · soak · scale-out · 5-min 3× burst · BDD · production cloud deployment
---	---	--

Technology

Go services · Kafka (KRaft) · PostgreSQL 16 + pgvector (RLS) · Redis · ClickHouse · Keycloak (OIDC) · Vault PKI (dev mode) · React + TypeScript + MapLibre GL · Python (scikit-learn) · Prometheus / Grafana / OpenTelemetry collector · Docker Compose, Helm, Terraform, GitHub Actions.

Contents

00	Why VoltSight deserves attention	2
01	Executive summary	4
02	Problem definition	4
03	Users and personas	4
04	VoltSight solution	4
05	Live application: dashboard, map, decision (Figures 5–8, 15–16)	6
06	System architecture	5
07	Data flow	7
08	Polyglot persistence	8
09	Database design	8
10	SQL optimisation (measured EXPLAIN ANALYZE)	9
11	Algorithms and data structures	9
12	Machine learning	10
13	Agentic AI: Charge Ops Copilot	11
14	Security	11
15	STRIDE threat model	12
16	Compliance and privacy	12
17	Observability	13
18	Reliability and distributed systems	13
19	Hot / warm / cold data lifecycle	14
20	Performance: target vs measured	15
21	Testing	15
22	CI/CD	15
23	Containerisation and deployment	16
24	Architecture decision records	16
25	Requirement traceability matrix	17
26	Measured boundaries & remaining engineering work	18
27	AI and open-source declaration	18
28	Reproducibility	18
29	Demo guide (5 minutes)	18
30	Conclusion	19

Page numbers are computed from the rendered PDF. Sections are numbered 00–30; Section 05 holds the live-application figures. Figures 5–8 (dashboard, map, decision) are on the pages listed above.

01 Executive summary

VoltSight is a multi-tenant platform for EV fleet operators. Vehicles stream telemetry over mutually authenticated TLS; a stream worker evaluates every event against a per-tenant graph overlay that knows the distance to the nearest *available* charger; an alert is raised, with its evidence and a route, when a vehicle can no longer reliably reach one. Dispatchers see the fleet on a live console and map, ask an audited Copilot, and approve charging plans. History feeds batch analytics (trips, battery state of health) and an ML consumption model.

The system was built and measured on a single 4-vCPU Linux host with every component co-located. It reached 98.3K events/s for 60 s with 100,000 vehicles, lost no events in five fault-injection scenarios, and serves API p95 26.8 ms / p99 41.0 ms. It did **not** meet the < 5 s critical-alert target at 100K events/s on that host (p50 9.8 s), nor was a soak, a 5-minute 3× burst or scale-out run. Section 26 states these boundaries and Section 25 gives all 60 requirements with status: **PASS 28, PARTIAL 28, BLOCKED 2, FAIL 2** (computed from the matrix file).

02 Problem definition

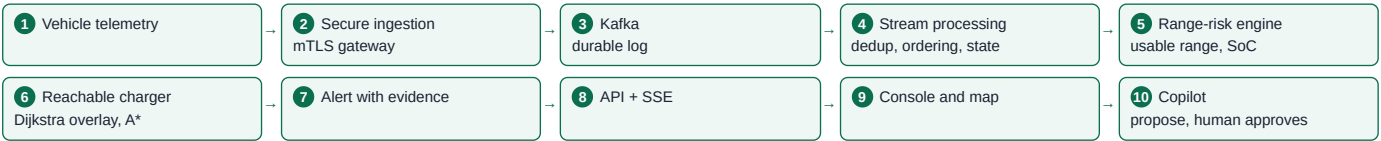
- **Operational problem.** A stranded EV costs a tow, a missed delivery and a driver. Range shown on the dashboard is nominal; usable range depends on the vehicle, route, temperature, load and battery age.
- **Range-risk intelligence.** The right question is not “what is the SoC” but “can this vehicle reach a *working* charger with margin, given where it is now”. The answer changes when a charger fails.
- **Charging cost.** Time-of-use tariffs make when and where to charge a large lever; uncoordinated plugging in at peak raises cost and depot congestion.
- **Battery health.** State of health is not reported by the vehicle; it must be inferred from charging sessions.
- **Decision latency.** At 100,000 vehicles reporting once per second the platform must take 100,000 decisions per second and surface the critical ones within seconds.

03 Users and personas

Persona	Goal	In VoltSight
Dispatcher	Keep vehicles from stranding	Dashboard, map, alert evidence, Copilot, acknowledge/resolve alerts (role <code>dispatcher</code>)
Fleet manager	Fleet-level visibility	Vehicles, reports, state-of-charge distribution (viewer / dispatcher roles)
Energy manager	Lowest-cost charging	Charge plans (DP optimiser), approval of plans (role <code>energy_manager</code>)
Tenant administrator	Governance and privacy	Audit trail, users, erasure workflow (role <code>tenant_admin</code>)
Platform engineer	Run the platform	Platform page, Prometheus/Grafana, health endpoints, Compose/Helm/Terraform

04 VoltSight solution

Telemetry → decision → evidence → action, as one closed loop.



Workflow — from telemetry to operator decision (each step is a real component; see Figure 1).

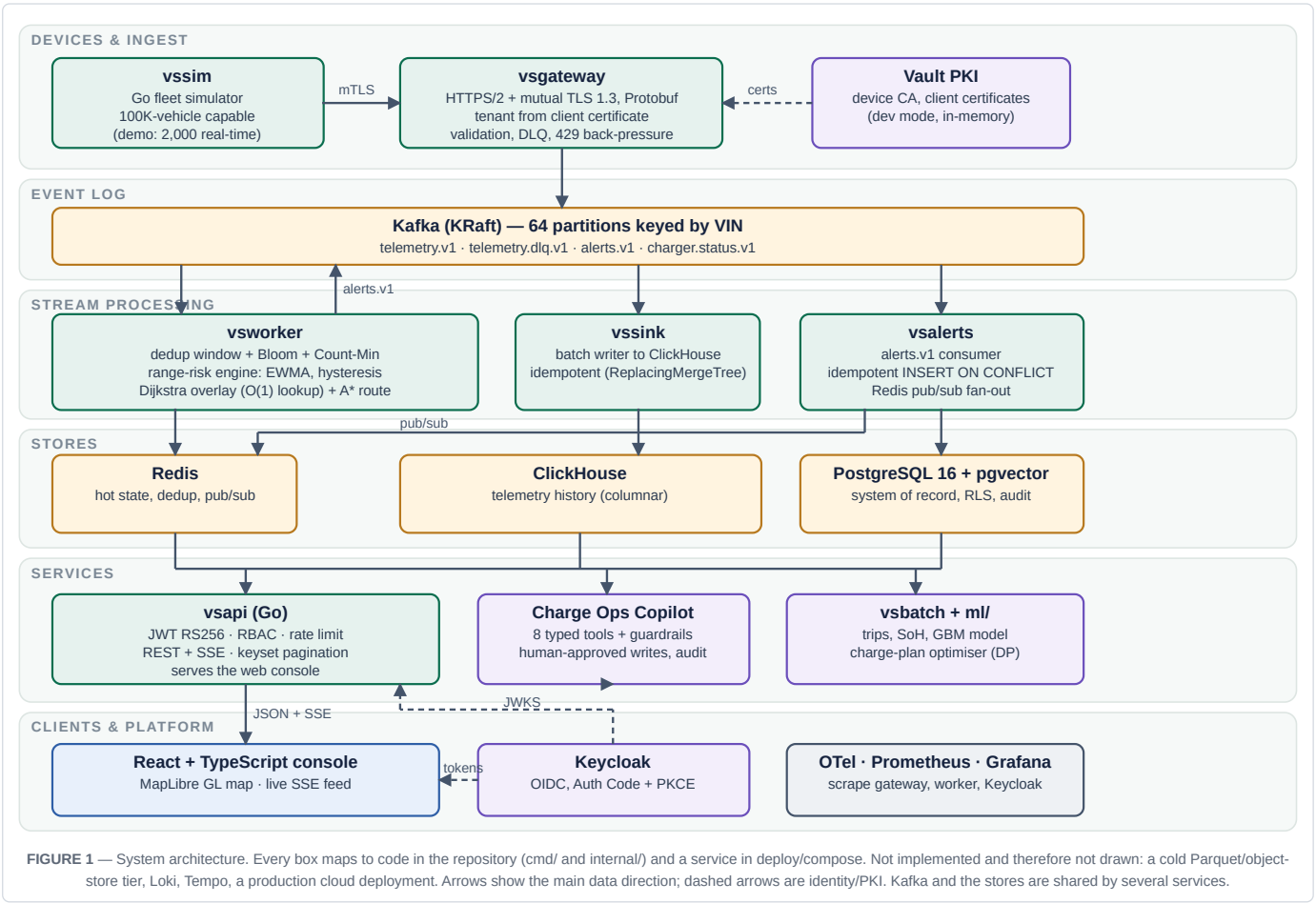
Business value

- **Prevent stranding** — risk is raised minutes ahead: in the simulator stress scenario the EWMA estimator caught 25 of 25 strandings with a 53 min median lead (precision 0.29, i.e. many early warnings; plain “SoC < 15%” caught 92% at precision 0.07).
- **Identify charging risk** — alerts carry usable range, margin, nearest available charger and route.
- **Charging cost** — the DP optimiser was cheaper than charge-on-plug-in in 389 of 392 random test cases (41.8% lower total cost on that synthetic tariff set). No real-world saving is claimed.
- **Battery health** — state of health inferred from charging sessions (simulator truth: MAPE 0.5%, optimistic by construction).
- **Operational decision support** — live console, Copilot answers with citations, approval-gated plans.

No monetary savings are stated: none were measured.

06

System architecture



Component	Repository	Role
vssim	cmd/vssim, internal/sim	Deterministic physics simulator (trips, charging, faults, duplicates, out-of-order, outages, two OEM dialects)
vsgateway	cmd/vsgateway, internal/ingest	HTTPS/2 + mTLS 1.3 ingest, tenant from certificate, validation, DLQ, back-pressure 429
vsworker	cmd/vsworker, internal/rtp, rangerisk, dedup	Dedup, state, range-risk engine, alerts
vssink / vsalerts	cmd/vssink, cmd/vsalerts	ClickHouse writer; idempotent alert persistence + Redis fan-out
vsapi + web	cmd/vsapi, internal/api, web/	REST + SSE, JWT/RBAC, console and map
vsbatch / ml	cmd/vsbatch, internal/batch, ml/	Trips, SoH, charge-plan optimiser, GBM model
Infra	deploy/compose	Kafka, PostgreSQL+pgvector, Redis, ClickHouse, Keycloak, Vault, Prometheus, Grafana, OTEL collector, Caddy

05

Live application: dashboard, map and decision

All screenshots are unedited captures of the VoltSight live profile (the same Compose profile that runs in Codespaces) taken on 2026-10-02 in the build environment, signed in as `dispatcher@meridian.example`. Numbers in captions are the values visible at capture time. The simulator runs the built-in deterministic “range-risk” demo scenario: 30% of vehicles start at 10–13% SoC and, five minutes in, 99% of the charger network is taken out of service for ~50 minutes.

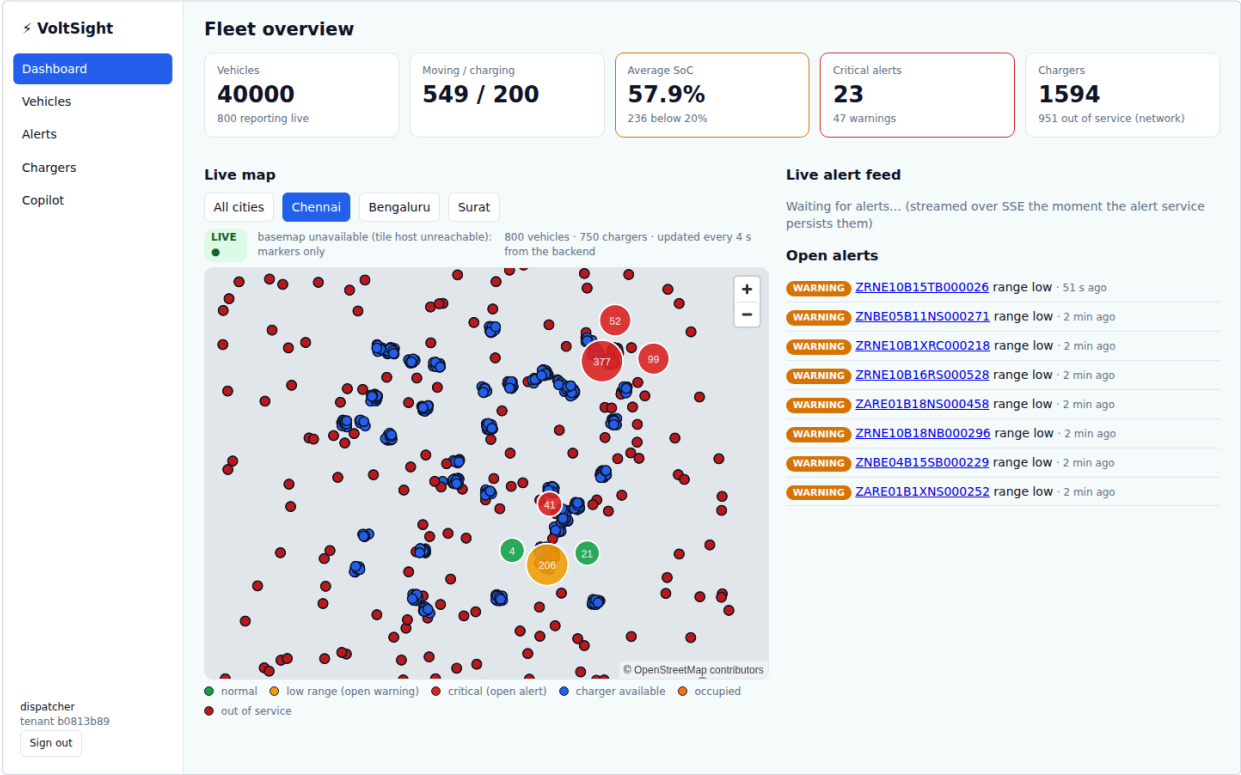


FIGURE 5 — Fleet overview with the live map filtered to Chennai: 800 vehicles reporting live (of 40,000 registered in the tenant) and 750 chargers, clustered markers coloured by backend risk state. The OpenFreeMap basemap host was unreachable from this capture environment, so the map shows the built-in markers-only fallback (stated on screen).

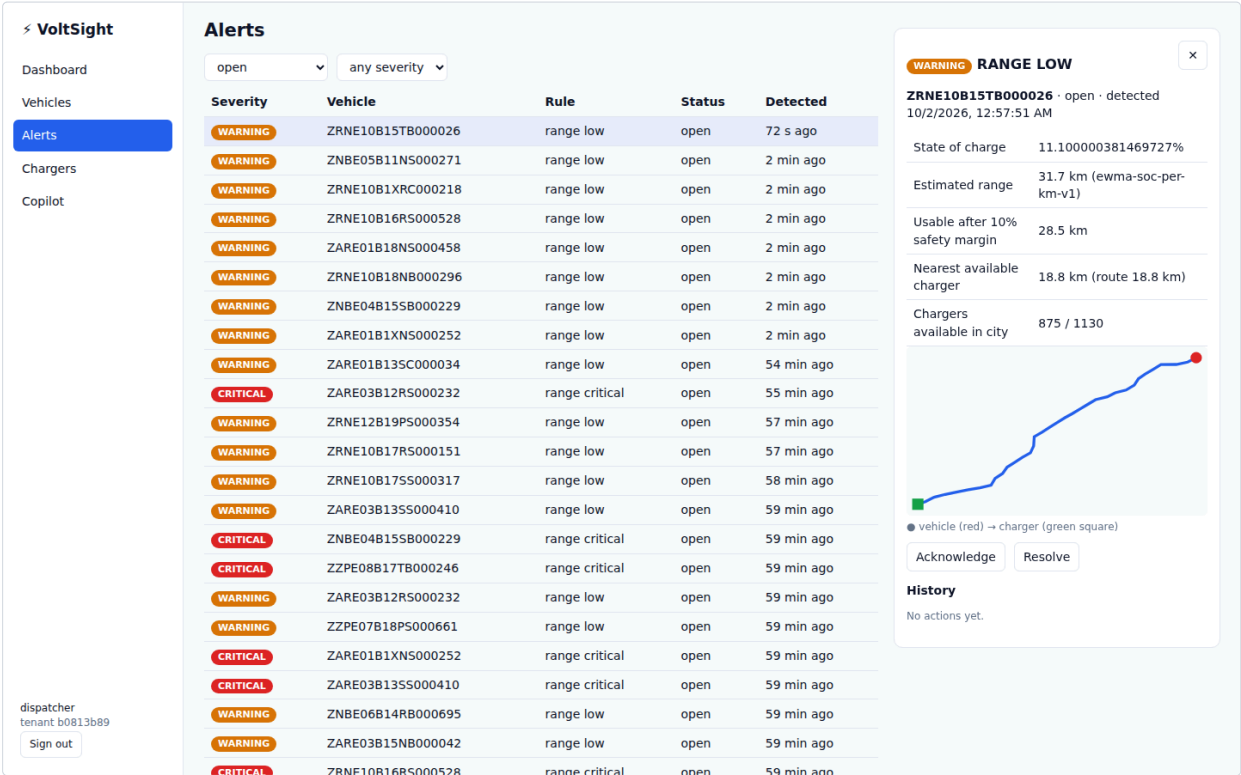


FIGURE 6 — Alert evidence: WARNING “range low” for one vehicle — SoC 11.1%, estimated range 31.7 km (EWMA estimator), 28.5 km usable after a 10% safety margin, nearest available charger 18.8 km, 875 of 1,130 chargers available in the city; the A* route from the vehicle (red) to the charger (green square) is drawn from the alert’s own evidence.

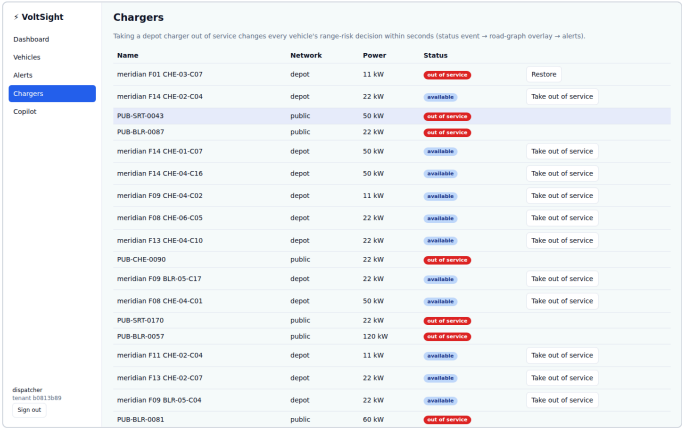


FIGURE 7 — Charger availability page (backend charger status).

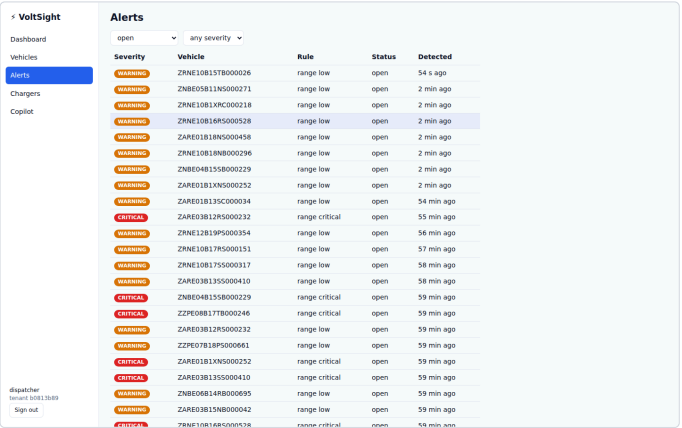


FIGURE 8 — Alert inbox sorted by detection time, with open WARNING and CRITICAL alerts raised by the risk engine from the telemetry.

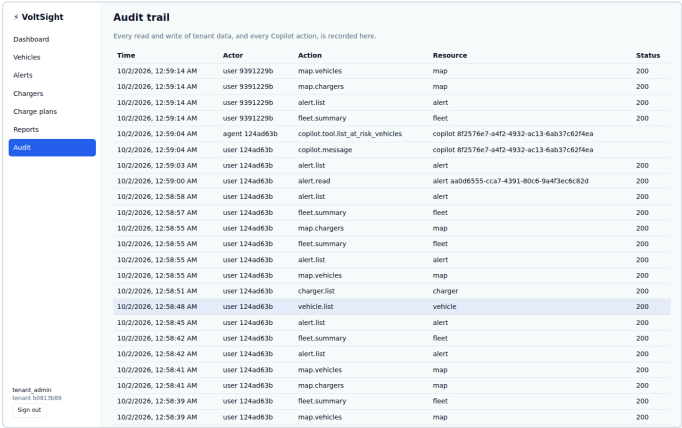


FIGURE 15 — Audit trail as tenant administrator: actor, action, resource and request id for data access and AI actions.

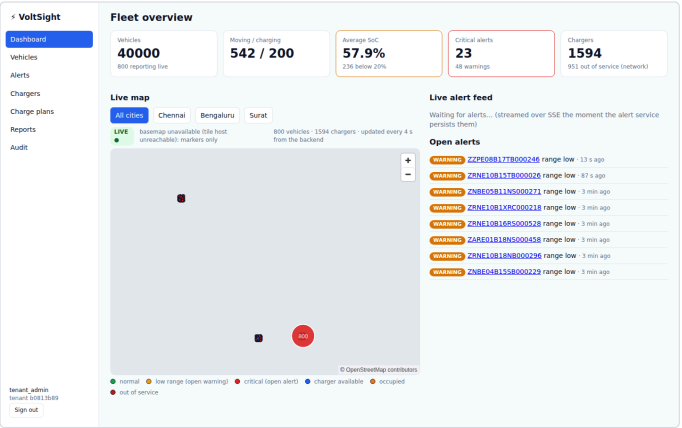


FIGURE 16 — Platform page for operators.

07 Data flow

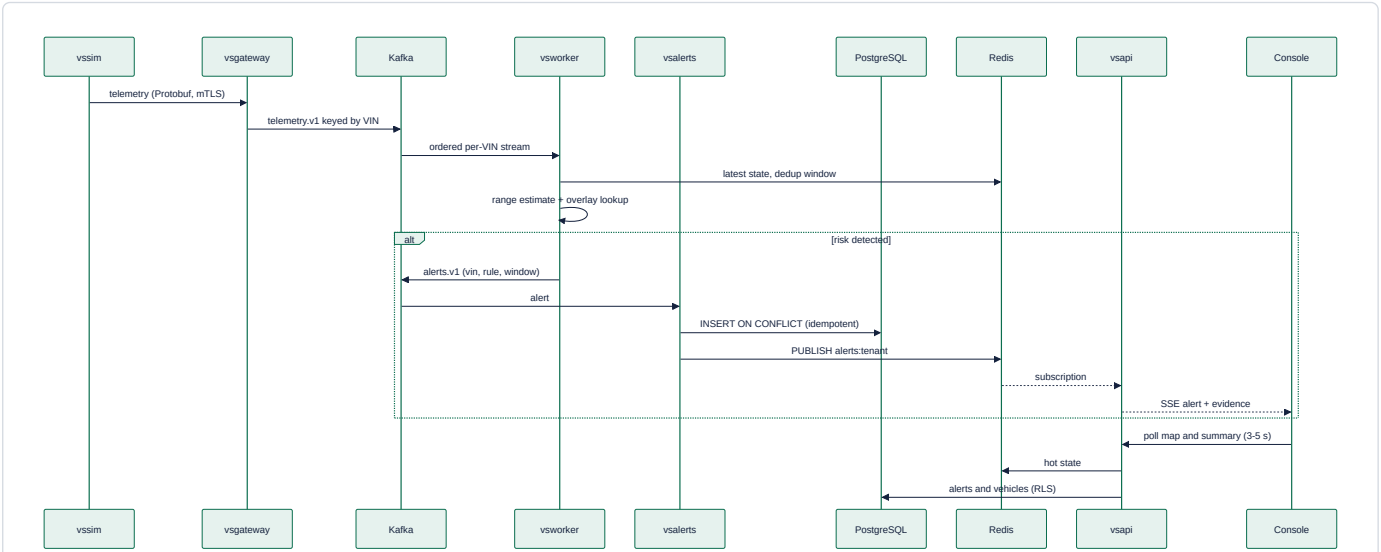


FIGURE 2 — Real-time hot path from device event to browser. Delivery is at-least-once with idempotent sinks (alert key (vin, rule, window_start); ClickHouse ReplacingMergeTree).

Stage	What happens	Evidence
Telemetry	Protobuf event with VIN, ts, lat/lon, speed, soc, odo, dtc, evt, seq (field names follow the problem statement's example)	evidence/G1
Validation	VIN check digit, DTC parse, schema checks; bad events go to a DLQ topic with a reason	internal/normalise
Schema evolution	Two OEM payload dialects and a schema roll-out are normalised to one contract; Protobuf buf breaking gate	internal/normalise, evidence/G1
Ordering	Kafka key = VIN keeps per-vehicle order within a partition (64 partitions)	ADR-001
Deduplication	Exact 256-bit sequence window per VIN + rotating Bloom negative cache	internal/dedup
Late events	Event-time handling with tolerance for out-of-order (3% injected in the E2E run)	internal/rtp
State and features	Latest state in Redis; EWMA energy per km; hysteresis	internal/state, rangerisk
Alert decision	usable range vs distance to nearest available charger from the overlay; deterministic alert window	internal/rangerisk

Stage	What happens	Evidence
Persistence and UI	alerts.v1 → vsalerts → PostgreSQL → Redis pub/sub → API SSE → browser	internal/alerts, api

08

Polyglot persistence

Store	Purpose / data	Why chosen	Consistency	Lifecycle	Failure behaviour
PostgreSQL 16	Tenants, vehicles, alerts, plans, audit, subscriptions	Relational integrity, RLS, ACID approvals	CP; ACID, single primary	Hot; audit monthly partitions; no purge implemented	Writers retry with capped back-off; offsets committed after persistence
pgvector	Incident embeddings (similar-incident search)	Semantic retrieval inside the system of record	As PostgreSQL	Hot	Copilot says "insufficient evidence" if no hits
Redis	Latest vehicle state, dedup window, pub/sub	Sub-millisecond operational access	AP; rebuildable from the log	Hot, TTL/volatile	Worker retries; restart scenario: 0 lost
Kafka (KRaft)	telemetry.v1, dlq, alerts.v1, charger.status.v1	Durable log, replay, decoupling	Per-partition order; at-least-once	Retention-bound	Broker-restart scenario: 0 lost
ClickHouse	Telemetry history for analytics	Columnar ingest and scans	AP; ReplacingMergeTree idempotency	Warm; no cold tier	Sink retries; restart scenario: 0 lost
Object store / Parquet	Cold archive	—	—	NOT IMPLEMENTED (designed only)	—

Why not one SQL database at this scale? The problem statement asks for 100,000+ events/s, i.e. 8.6 billion events per day. Measured on the same host with 1M rows: PostgreSQL COPY 97K rows/s versus ClickHouse 405K rows/s (4.2×), and 17.3 B/row in ClickHouse for simulated telemetry. Operational state needs sub-millisecond reads (Redis) and transactional guarantees for approvals and audit (PostgreSQL). Evidence: `evidence/G14/pg_vs_clickhouse.txt`, ADR-002.

09

Database design

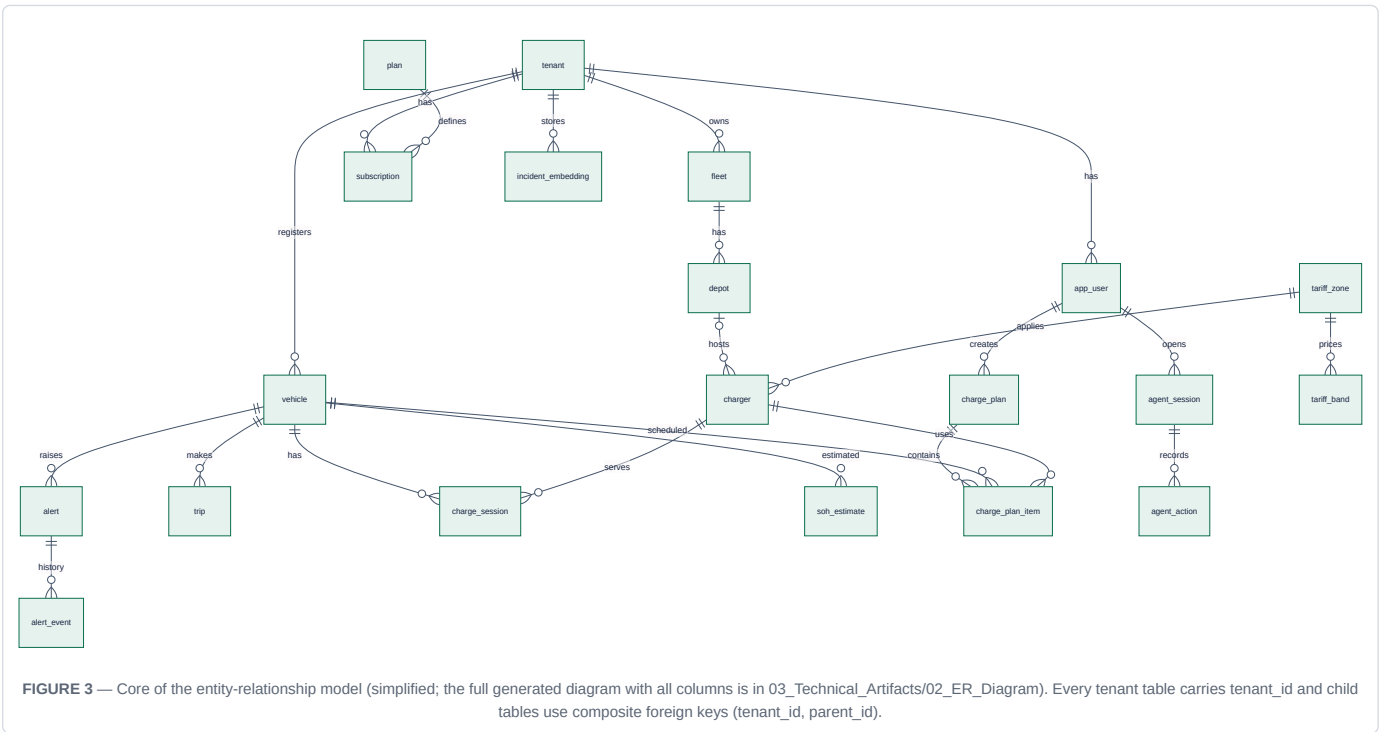


FIGURE 3 — Core of the entity-relationship model (simplified; the full generated diagram with all columns is in 03_Technical_Artifacts/02_ER_Diagram). Every tenant table carries `tenant_id` and child tables use composite foreign keys (`tenant_id, parent_id`).

- **3NF** — with one documented, deliberate denormalisation: `tenant_id` is repeated on child tables so that composite FKs make a cross-tenant reference impossible; the FK keeps it from drifting (ADR-006, `docs/3nf.md`).
- **RLS** — enabled and forced on every tenant table, policy `tenant_id = current_setting('app.tenant_id')::uuid` (fails closed when unset); no application role has `BYPASSRLS` or `DDL`.
- **Partitioning / indexes** — audit log in monthly partitions pruned by `ts`; partial index for the open-alert inbox; composite indexes for trips and audit lookups.
- **Tests** — RLS attack suite on every tenant table (cross-tenant read/insert/update, unset and malformed GUC, joins/views), concurrent plan approval has exactly one winner in 100 rounds.

10 SQL optimisation (measured EXPLAIN ANALYZE)

Real plans on a database with 1.5M alerts, 1.5M trips and 600K audit rows for one tenant, parallelism disabled for comparability ([evidence/G9b/plans/sqlopt.txt](#), [tools/sqlopt/bench.sql](#)).

#	Query	Change	Before	After
1	Alert inbox, deep page	OFFSET 30,000 → keyset (detected_at, id) < + partial index WHERE status='open'	1,306 ms (OFFSET); 270 ms (keyset, no index)	0.28 ms
2	Vehicle list with open-alert counts	Per-row subquery (N+1) → one LEFT JOIN LATERAL query	—	8 statements per request for 5 and for 200 rows (test)
3	Trips of a vehicle in a window	Composite index (tenant_id, vin, started_at DESC)	0.67 ms	0.19 ms
4	Daily energy report	Aggregate over 1.5M trips → materialised view trip_daily	1,310 ms	0.024 ms
5	Audit by actor, last hour	Composite index; monthly partition pruning	0.68 ms	0.28 ms
6	RLS predicate form	app_tenant() vs (SELECT app_tenant())	10.2 ms	9.9 ms — no meaningful difference (reported as measured)

BEFORE — OFFSET pagination (1,306 ms)

```
Limit (actual time=1298.186..1298.196 rows=50 loops=1)
  Buffers: shared hit=64 read=31381, temp read=7933 written=14413
  -> Sort (actual time=1284.138..1287.799 rows=30850 loops=1)
    Sort Key: detected_at DESC, id DESC
    Sort Method: external merge  Disk: 57560kB
    Buffers: shared hit=64 read=31381, temp read=7933 written=14413
    -> Seq Scan on alert (actual time=0.161..615.813 rows=740097 loops=1)
      Filter: ((status = 'open'::text) AND (tenant_id = (current_setting('app.bench_tenant')::text)::uuid))
      Rows Removed by Filter: 760822
      Buffers: shared hit=64 read=31381
Planning:
  Buffers: shared hit=31
Planning Time: 0.288 ms
Execution Time: 1306.380 ms
```

AFTER — keyset + partial index (0.28 ms)

```
Limit (actual time=0.260..0.260 rows=0 loops=1)
  Buffers: shared read=3
  -> Index Scan using alert_inbox_open_idx on alert (actual time=0.255..0.255 rows=0 loops=1)
    Index Cond: ((tenant_id = (current_setting('app.bench_tenant')::text)::uuid))
    Buffers: shared read=3
Planning:
  Buffers: shared hit=66 read=2
Planning Time: 0.457 ms
Execution Time: 0.280 ms
```

Honest note: the AFTER probe's cursor returned 0 rows (rows=0 in the plan), so 0.28 ms is the cost of the index seek, not of materialising a full 50-row page. The BEFORE/AFTER ratio therefore overstates a typical page; the plan shapes (Seq Scan + external sort vs Index Scan) are the evidence.

11 Algorithms and data structures

Only algorithms that exist in the repository and are used (or explicitly flagged as not yet used). Source: [docs/algorithms.md](#), [internal/rangerisk](#), [internal/dedup](#), [internal/chargeplan](#), [internal/batch](#), [internal/geo](#).

Algorithm	Problem / job in the product	Complexity	Verification and measurement
Multi-source reverse Dijkstra (binary heap) over a 48,400-node CSR road graph	distance from every node to the nearest <i>available</i> charger usable by a tenant, per (city, tenant)	$O((V+E) \log V)$ rebuild, $O(V)$ space	property test vs an independent $O(V^2)$ Dijkstra on 120 random graphs (incl. disconnected, co-located sources). MEASURED (box): full rebuild 10.1 ms (48K nodes, 300 chargers)
Incremental overlay update (remove: invalidate the charger's catchment, re-seed from valid neighbours; add: relax from the new source)	charger outage/restore reflected in decisions without a rebuild	proportional to the catchment	60 random status walks × 40 steps == full rebuild. MEASURED (box): remove+add 70.7 μs
$O(1)$ lookup	per-event decision	$O(1)$	MEASURED (box): 3.6 ns/op, 0 allocs
A* (admissible straight-line heuristic)	route polyline in alert evidence	$O(E \log V)$ worst	equals Dijkstra length on 80 routes. MEASURED (box): 3.4× fewer node expansions (47,644 vs 161,701)
Geohash + $O(1)$ node snap	position → graph node; map cells; depot clustering	$O(1)$	tests; neighbours validated across cell borders
Per-VIN exact sequence window (256-bit bitmap)	dedup without false drops, tolerates reordering	$O(1)$ /event, 32 B/VIN	property test vs a reference set; MEASURED (Win): 4.3 ns/event
Rotating Bloom filter	negative cache for events older than the window	$O(k)$ /event, ~10 bits/element	MEASURED (Win): false-positive rate 0.0096 vs 0.0100 design; no false negatives; a positive is never used to drop
Count-Min sketch + heap	top-K diagnostic codes	$O(d)$ /update	test: never under-counts, 0 keys over $\epsilon \cdot N$, top-10 recall ≥ 9/10
EWMA of %SoC per km	per-vehicle consumption → range	$O(1)$ /event	decision-level evaluation (G6.9)
Dynamic programming over (slot × energy level) with a charge-curve taper and charger-occupancy mask	minimum-cost charging under time-of-use tariffs	$O(T \cdot S \cdot A)$ per vehicle	property test: feasible and never more expensive than charge-on-plug-in (392 random cases; strictly cheaper in 389; MEASURED total cost reduction 41.8% on random synthetic tariffs). MEASURED (box): 0.53 ms per vehicle (14 h, 15-min slots). Synthetic depot: 45% cheaper than plug-in charging
Stop/trip segmentation (speed hysteresis + dwell)	GPS/speed → trips	$O(n)$	MEASURED (simulator truth, box): precision 0.911, recall 1.000, F1 0.954 on 1,962 trips
Douglas-Peucker	polyline simplification	$O(n \log n)$ avg	unit test (shape and end points kept) — not used in the live UI yet
Union-Find on geohash cells	clusters of stops at unapproved locations	$O(n \cdot \alpha(n))$	planted-cluster test; run over live data not evaluated
VIN check digit / DTC parser	normalisation	$O(1)$	exhaustive DTC round trip; VIN property tests
Keyset pagination	API lists	$O(\log n)$ seek	<code>TestKeysetPagination</code> : 0.28 ms vs 1.306 ms for OFFSET (docs/database/SQL_OPTIMIZATION.md)
Feature-hashing embedder + HNSW cosine	similar-incident search	$O(\text{len})$ embed; HNSW $\sim O(\log n)$	recall@5 0.958 on a labelled paraphrase set (lexical; limitation)
Gradient-boosted trees (scikit-learn HistGB)	trip consumption	—	MAPE 7.17% vs 11.52% best baseline (vehicle-level hold-out), paired-bootstrap CI excludes 0 (evidence/G8)

Range-risk decision (pseudocode)

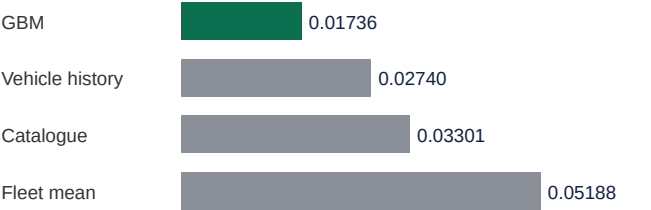
```
on event e for vehicle v (tenant t):
  state[v] = merge(state[v], e)                # Redis, per-VIN dedup first
  rate    = EWMA(kWh or %SoC per km)          # 0(1)
  usable  = remaining_energy / rate * (1 - safety_margin)
  d       = overlay[t][node(e.lat, e.lon)]      # 0(1): distance to nearest AVAILABLE charger
  if usable < d * road_factor: risk = CRITICAL / WARNING (with hysteresis)
  emit alert(vin, rule, window_start) with evidence {soc, usable, d, route = A*(node, charger)}
on charger status change: update overlay incrementally (70.7 μs)
```

12

Machine learning

Item	Value
Problem	Predict trip energy consumption (kWh/km) to improve range estimates
Features	Vehicle model/age, speed and route profile, temperature, trip length (payload, driver style and true SoH are hidden)
Baselines	Fleet mean, model catalogue, per-vehicle history (best)
Model	HistGradientBoostingRegressor , sha256 aef157a2...
Data	Simulated: 2,500 vehicles, 24,006 trips
Split	A: 500 held-out vehicles (4,755 trips). B: time hold-out (4,813 trips)
Leakage	Vehicle sets disjoint; features exclude label; first-trip prior = catalogue; shuffled-label model MAE 0.0515 (worse)

Split A — MAE kWh/km (lower is better)



Split B — MAE kWh/km (lower is better)

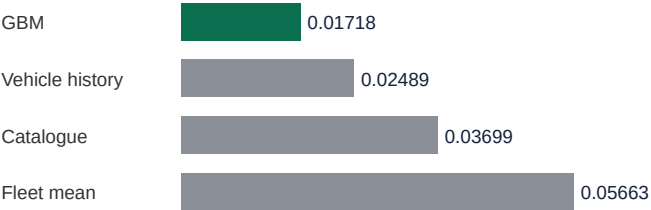


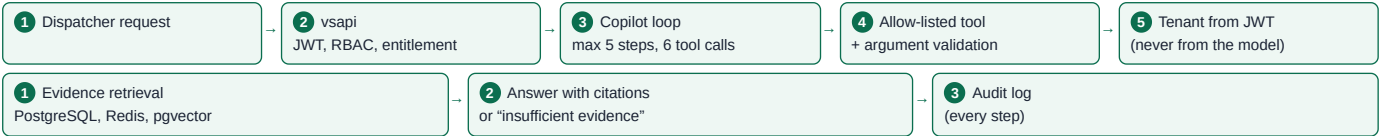
FIGURE 4 — MAE of the GBM against three baselines on two hold-outs, drawn from evidence/G8/ml_report.json .

Split	GBM MAE / MAPE	Best baseline MAE / MAPE	95% CI of difference
A vehicle hold-out	0.01736 / 7.17%	0.0274 / 11.52% (vehicle history)	−0.01124 ... −0.00883
B time hold-out	0.01718 / 6.60%	0.02489 / 9.86% (vehicle history)	−0.00862 ... −0.00680

Decision impact: the estimate feeds the range-risk engine’s consumption term; the live engine uses the EWMA estimator, the GBM is a batch artifact (a Go evaluator was not built). **Limitations:** labels come from the simulator’s physics, so absolute errors do not transfer to real fleets; no drift monitoring.

13 Agentic AI: Charge Ops Copilot

Read path (answers)



Write path (actions)

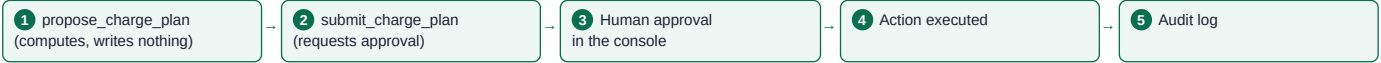


FIGURE 12 — Copilot control flow. Policy is enforced in code outside the model; the agent cannot approve its own proposal.

Tool	Effect
list_at_risk_vehicles / get_vehicle_state / get_alert_evidence	read tenant data
find_reachable_chargers	read; straight-line distance with a 1.3 road factor (baseline estimate, not routed)
query_telemetry_summary	read; aggregated telemetry for a vehicle or the fleet
search_similar_incidents	read; pgvector similarity — returned text is treated as data, not instructions
propose_charge_plan	computes a plan, writes nothing
submit_charge_plan	requests approval; the agent cannot approve — a human does in the console

- **Guardrails:** tool allow-list; argument validation; tenant from the authenticated identity; 1–1000 char messages with unknown fields rejected; 5 steps / 6 tool calls; citations or “insufficient evidence”; two-phase writes; every step audited; viewers and non-enterprise tenants get 403. Injection and cross-tenant scenarios: `evidence/G10` (8 scenarios).
- **Provider (stated precisely):** the default is a deterministic stub, used for all recorded scenarios and screenshots. The Anthropic provider activates only with `ANTHROPIC_API_KEY` and was **not exercised**; no claim is made about LLM answer quality.

VoltSight

Dashboard

Vehicles

Alerts

Chargers

Copilot

Charge Ops Copilot

Answers come only from tool calls on your tenant's data (with your permissions); every step is audited. Writes need your approval.

Which vehicles are at risk right now?

10 vehicle(s) with open range-risk alerts:

- ZARE03B12RS000232 — CRITICAL, SoC 7.4%, usable range 18.2 km, nearest available charger 18.8 km [alert:b34cef98-b302-4b7d-a493-dcb39bf1083b]
- ZZPE08B17TB0000246 — CRITICAL, SoC 5.2%, usable range 13.9 km, nearest available charger 18.8 km [alert:8e29eff5-be5a-4eb3-b2be-4f0d4e4c1eaf]
- ZNBE06B14SC0000373 — CRITICAL, SoC 7.9%, usable range 19.2 km, nearest available charger 18.8 km [alert:42882d18-6b0e-4bc6-b169-ec14a888b0df]
- ZRNE10B15TB000026 — WARNING, SoC 11.1%, usable range 28.5 km, nearest available charger 18.8 km [alert:aa0d6555-cca7-4391-80c6-9a4f3ec6c82d]
- ZNBE05B11NS000271 — WARNING, SoC 9.6%, usable range 28.7 km, nearest available charger 18.8 km [alert:3e1e917c-44b4-40f4-a92b-373e6a8be014]
- ZRNE10B1XRC000218 — WARNING, SoC 10.2%, usable range 26.2 km, nearest available charger 18.8 km [alert:25d70632-87b2-4693-86d2-34f17e24ef0e]
- ZRNE10B16RS000528 — WARNING, SoC 10.7%, usable range 27.5 km, nearest available charger 18.8 km [alert:dcbb19a-d1e4-4e9a-8f7b-155fb8407a0d]
- ZARE01B18NS000458 — WARNING, SoC 10.9%, usable range 28.4 km, nearest available charger 18.8 km [alert:b4a334f4-4b46-486d-a2d5-9a4c73a53426]
- ZRNE10B18NB000296 — WARNING, SoC 9.5%, usable range 24.4 km, nearest available charger 18.8 km [alert:2986a10a-e6c4-4bd9-864f-a988716ea0f8]
- ZNBE04B15SB000229 — WARNING, SoC 10.6%, usable range 26.3 km, nearest available charger 18.8 km [alert:2551dd3b-d3d7-4bf8-9fcc-874aa280d627]

sources: alert:2986a10a-e6c4-4bd9-864f-a988716ea0f8, alert:b34cef98-b302-4b7d-a493-dcb39bf1083b, alert:8e29eff5-be5a-4eb3-b2be-4f0d4e4c1eaf, alert:3e1e917c-44b4-40f4-a92b-373e6a8be014, alert:dcbb19a-d1e4-4e9a-8f7b-155fb8407a0d, alert:b4a334f4-4b46-486d-a2d5-9a4c73a53426, alert:2551dd3b-d3d7-4bf8-9fcc-874aa280d627, alert:42882d18-6b0e-4bc6-b169-ec14a888b0df, alert:aa0d6555-cca7-4391-80c6-9a4f3ec6c82d, alert:8e29eff5-be5a-4eb3-b2be-4f0d4e4c1eaf

Ask about at-risk vehicles, alerts, chargers, plans...

Send

dispatcher

tenant b0813b89

Sign out

FIGURE 13 — Copilot answering “Which vehicles are at risk right now?” — each line cites an alert id; sources are listed. Stub provider, real data from the running deployment.

14 Security

User path



Device path



FIGURE 14 — Security architecture: user path and device path.

Control	Implementation	Evidence
Authentication	OIDC Authorization Code + PKCE; password grant refused; DEV-only client absent from the public realm	live_checks EXP-3/4
Session	300 s access token renewed by refresh-token grant; single-flight; one retry on 401; SSE reconnect with fresh token	web/src/auth.test.ts (14 tests)
Authorization	RBAC roles + subscription entitlements; viewer cannot ack alerts, use Copilot or read audit	live_checks RBAC-1...6

Control	Implementation	Evidence
Tenant isolation	Token claim → RLS; cross-tenant reads return 404 (no existence leak)	live_checks ISO-1...4; RLS suite
Transport	Devices mTLS 1.3; browsers TLS at edge; HSTS, CSP, nosniff, DENY framing	live_checks EXP-5
Rate limiting / input	Per-client rate limit; body-size limits; VIN/DTC/schema validation; strict JSON	internal/api, internal/ingest
Secrets	Generated per deployment into a git-ignored .env; gitleaks clean; Vault dev mode (in-memory)	evidence/G12
Audit	Append-only audit_log with request id and hash chain	internal/auditchain
Edge exposure	Keycloak admin, master realm and metrics return 404 at the proxy	live_checks EXP-1/2

Scan evidence

Scan	Result	File
gitleaks / govulncheck / npm audit / semgrep / Trivy fs	clean (semgrep: 1 finding waived)	evidence/G12/status.md
Trivy image	0 HIGH/CRITICAL	evidence/live-deployment/image-scan.txt
SBOM	CycloneDX generated for the image	evidence/security/sbom-image.cdx.json
OWASP ZAP 2.17.0 baseline (DAST)	58 passed, 0 failures, 3 warning groups (CSP: directive with no fallback; CSP style-src unsafe-inline; Unix timestamp disclosure; informational: modern web application)	evidence/security/zap/

ZAP scope: unauthenticated surface only; no authenticated or active scan; it ran **before** the CSP was tightened and was not re-run. Warnings are not failures, and they are not hidden.

15

STRIDE threat model

Threat	Asset	Attack	Mitigation	Residual risk	Evidence
Spoofing	Device identity	Fake device posts telemetry	mTLS 1.3, Vault-issued cert per tenant×OEM, VIN → tenant check	Dev-mode Vault in the demo	evidence/G12, ADR-004
Spoofing	User session	Stolen/forged JWT	RS256 + JWKS, short lifetime, PKCE	No token binding	live_checks SEC-1/2
Tampering	Telemetry / alerts	Replay or duplicate events	Per-VIN window dedup, idempotent alert key	Late events beyond the window	internal/dedup
Tampering	Audit log	Edit or delete history	Append-only, hash chain, no UPDATE/DELETE for app role	Chain verified offline only	internal/auditchain
Repudiation	Approvals / AI actions	"I never approved"	Audit rows with actor, request id; Copilot cannot approve	—	evidence/G10
Information disclosure	Other tenants' data	IDOR / SQL path	RLS forced, composite FKs, 404 on cross-tenant	Caller-controlled GUC is a documented limitation	evidence/G1, STRIDE doc
Information disclosure	Vehicle location	Viewer reads precise GPS	Role-based masking	—	internal/privacy, api tests
Denial of service	Gateway, API	Flood	Back-pressure 429, rate limit, bounded bodies	No load-balanced DDoS protection	G3, G9
Elevation of privilege	Copilot	Prompt injection to act	Allow-list, validation, tenant from JWT, human approval, retrieved text is data	Real LLM not exercised	evidence/G10
Elevation of privilege	Keycloak admin	Reach admin console	404 at edge for /admin, master realm	—	live_checks EXP-1/2

Full model: docs/security/STRIDE.md (including the documented residual risks).

16

Compliance and privacy

- **Location masking** by role (geo.precise permission); the map and API return reduced precision to roles without it.
- **Audit** of data access and AI actions with request id and a verifiable hash chain.
- **Right to erasure** — an erasure workflow (vsprivacy) with its own database role; tested.
- **Retention** — Kafka and ClickHouse retention are configuration; a PostgreSQL audit-retention purge is **not implemented**.
- **GDPR, DPDP, UNECE R155/R156** are mapped at awareness level in docs/compliance.md . No legal compliance or certification is claimed. All data is synthetic.

17

Observability

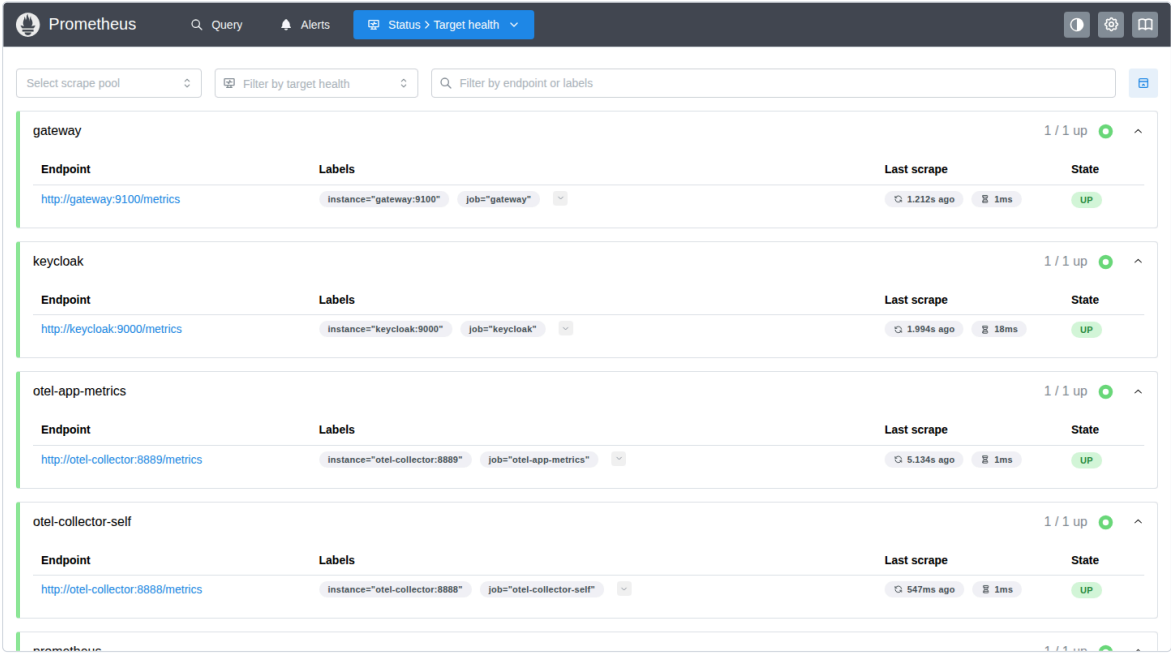


FIGURE 10 — Prometheus targets page of the running deployment: gateway, worker, keycloak, otel-collector and Prometheus scrape targets all up at capture time.

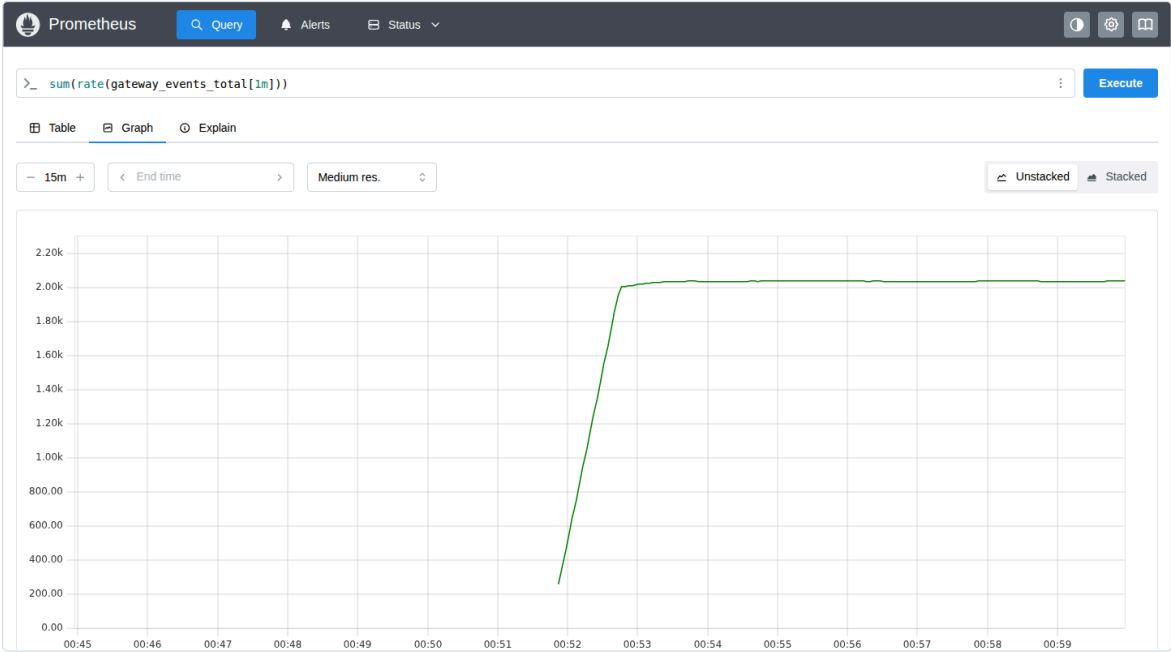


FIGURE 11 — Prometheus graphs of gateway events/s and worker consumer lag (rtp_consumer_lag_records) at capture time (demo scale, not the 100K load test).

- **Logs** — structured `slog`; API errors carry `request_id` (also stored in `audit_log.request_id`).
- **Metrics** — Prometheus `/metrics` on gateway and worker (accepted/rejected events, consumer lag, ingest-process histogram), Keycloak; scrape interval 5 s.
- **Traces** — OpenTelemetry collector deployed; end-to-end distributed tracing is **not** fully instrumented; no Tempo or Loki backend.
- **Alert latency** — alert service logs receive → persist and receive → decision per alert; Kafka lag and throughput via the reconciliation tool `vsrecon`.
- Grafana is provisioned with the Prometheus datasource; no Grafana dashboard screenshot is included.

18

Reliability and distributed systems

Topic	Position	Status
CAP / PACELC	PostgreSQL (approvals, audit) CP; telemetry/state AP and rebuildable. During partition favour availability of ingest; else latency over consistency for hot state. docs/architecture/CAP_PACELC.md	Documented
Partitioning	64 Kafka partitions keyed by VIN; audit table partitions by month	Implemented
Replication	Single broker/primary in Compose; RF and replicas are Helm/Terraform parameters	Designed; not exercised
Delivery semantics	At-least-once end to end + idempotent sinks ⇒ effectively-once observable results. Kafka exactly-once transactions are not used (ADR-003)	Implemented
Idempotency	Alert key (vin, rule, window_start); ClickHouse ReplacingMergeTree; per-VIN dedup	Implemented
Back-pressure	Gateway returns 429 + Retry-After; consumers pull at their own rate	Implemented
Circuit breaker	Bounded retries with capped exponential back-off; no circuit-breaker library	Partial
Cache invalidation	Redis state overwritten per event; overlay updated incrementally on charger status	Implemented; bound not tested

Topic	Position	Status
Horizontal scale	Stateless workers scale with partitions; no scale-out experiment was run	Designed; NOT VERIFIED
Graceful degradation	Map falls back to markers-only if the basemap host is unreachable; services retry instead of exiting	Implemented
CQRS / event sourcing	Kafka log is the source for read models (ClickHouse, Redis); no formal event-sourced aggregate	Partial

19

Hot / warm / cold data lifecycle

Tier	Where	Status
Hot	Redis (latest state), PostgreSQL (operational), Kafka (log)	IMPLEMENTED
Warm	ClickHouse history (29.2M rows benchmarked, 129–720 ms queries)	IMPLEMENTED
Cold	Parquet on object storage with lifecycle policy and cost estimate	NOT IMPLEMENTED — DESIGNED ONLY

20

Performance: target vs measured

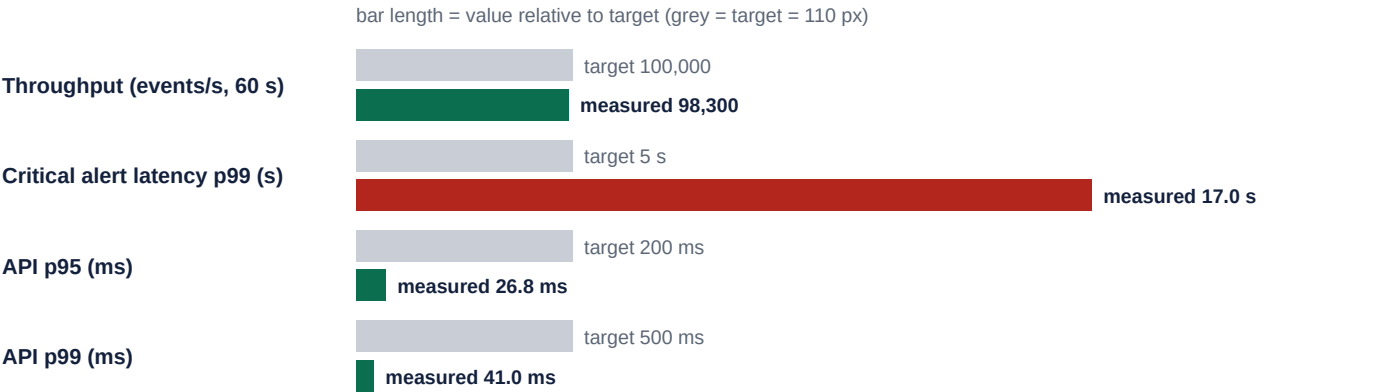


FIGURE 9 — Targets from the problem statement against measurements. Alert latency is receive → persisted at 100K events/s (p99); the other three are met or within 2%.

Metric	Target	Measured	Duration	Environment	Status
Vehicles	100,000	100,000 streamed	60 s	4 vCPU, all components on one host	PASS
Events/s	100,000+	98.3K (6.0M events, 0 failed, reconciled)	60 s	same	PARTIAL
3× burst, 5 min, no loss	3× for 5 min, 0 lost	3× for 40 s inside a 100 s run: 18.0M events, 0 lost, no 429s	40 s	same	PARTIAL
Critical alert latency	< 5 s	p50 9.8 s, p99 17.0 s receive → persist at 100K ev/s. Worker-only ingest → process: mean 1.9 s, all ≤ 5 s. Demo scale: receive → detect p50 28 ms, p95 133 ms (n=33)	100 s	same (CPU-saturated host)	FAIL
Dashboard latency	< 2 s	NOT VERIFIED end to end (map/summary polls 3–5 s; alerts pushed on SSE)	—	—	PARTIAL
API p95	< 200 ms	26.8 ms (p50 16.9) at 829 req/s, 16 clients, 100K-vehicle DB, ingest idle	20 s	same host	PASS
API p99	< 500 ms	41.0 ms	20 s	same host	PASS
Consumer lag	bounded	worker lag ≤ 5.3K records during the 98.3K ev/s run	60 s	same	PASS
Simulator throughput	—	demo profile ≈ 2,000 vehicles in real time; 100K-vehicle runs reached 98.3K ev/s through the whole pipeline	—	—	—
Soak / scale-out	required by statement	NOT RUN	—	—	PARTIAL

Why the alert target was missed (as attributed in evidence/G14): the single 4-vCPU host runs the load generator, gateway, Kafka, worker, sink, alert service, ClickHouse, Redis and PostgreSQL together; the latency is attributed to that oversubscription (gateway batch handling, Kafka acks, shared CPU), not to consumer backlog. In a separate 70 s worker-only run (no sink or alert service) receive → process was mean 1.9 s, 100% ≤ 5 s at 103K events/s with ~0 lag. No multi-host run was made, so whether the target is met on separate hardware is **NOT VERIFIED**. Details and storage calculations: docs/performance/CAPACITY_ANALYSIS.md (MEASURED / CALCULATED / DESIGN TARGET / NOT VERIFIED).

21

Testing

Category	Status	Evidence
Unit (Go)	PASS run; coverage gated ≥ 80% on the G1 packages (vin 100%, jwtverify 93%); later packages not measured	evidence/G1/status.md
Unit (web)	PASS 14 token-lifecycle tests	web/src/auth.test.ts
Integration / Testcontainers	PASS real Kafka, PostgreSQL, ClickHouse, Redis, Keycloak, Vault; RLS attack suite	evidence/G1
E2E	PASS 100K-vehicle live run; 31/31 scripted checks as real users; real-browser journeys	evidence/G4, live_checks.json
Contract	PARTIAL Protobuf round-trip and buf breaking (with negative test); no Pact	evidence/G1
BDD	FAIL not implemented	—
Load	PASS 100K vehicles 60 s; 3× burst 40 s	evidence/G4, G14
Soak	PARTIAL not run	—
Security	PASS SAST, dependency, secret, image scans; DAST baseline (partial)	evidence/G12, security/zap
Chaos	PASS 5 scenarios, 0 lost; Kubernetes pod kill and gateway kill not run	evidence/G13
Compliance	PASS audit trail and erasure tests	evidence/G1, G9

22

CI/CD

Workflow / job	What it does
ci / static checks	gofmt, vet, staticcheck, gitleaks, semgrep, buf lint/breaking, yamllint
ci / gate G1	contracts, database migrations, RLS attack suite, identity/PKI, 100K seed (real containers)
ci / application	build, unit tests, web tests, Helm lint + kubeconform, Terraform validate, requirement traceability gate
ci / container image	build, non-root check, Trivy scan; publish to GHCR on main
kubernetes	chart accepted by an ephemeral kind API server (dry-run, install, uninstall)

CI (GitHub Actions) — the branch head commit aa76192 passed workflow “ci” (runs 47 push and 48 pull_request, both success). Intermediate commits on the branch (runs 42–46) failed while the traceability gate, web tests and docs were being brought in, and were fixed forward; they are not hidden here. Run URLs: github.com/SohamB4746Y/voltsight-connected-vehicle-intelligence/actions (runs 36943593804 and 36943868217). No GitHub UI screenshot is included: the repository is private and the UI needs a login.

The tag v1.0-submission exists locally only (the push from the build environment was refused with HTTP 403) and points to an earlier commit; it was not moved.

23

Containerisation and deployment

Target	What exists	Status
Docker image	One 74 MB non-root image with all services and the console; built by CI and published to GHCR; Trivy 0 HIGH/CRITICAL	PASS validated
Docker Compose	<code>make up (dev)</code> and <code>deploy/live/up.sh</code> (single HTTPS origin, whole platform + simulator)	PASS deployed locally
Live demo	Same live profile run in GitHub Codespaces (public URL provided by the owner; stops when idle). A live demo environment — not a production cloud deployment	PARTIAL
Kubernetes / Helm	Chart <code>deploy/helm/voltsight</code> : lint + kubeconform; accepted by an ephemeral kind API server in CI; pods not run with all backing services	PARTIAL validated
Terraform	AWS and GCP modules: validate only; never applied	PARTIAL validated
Cloud portability	Everything runs from one image and environment variables; no cloud-specific code	Design

Free-tier hosting research and the dated limits: `docs/deployment-live.md`, ADR-010.

24

Architecture decision records

ADR	Problem	Decision	Alternative / trade-off	Evidence
001 Kafka	100K events/s with replay	Partitioned durable log keyed by VIN	Direct DB writes: contention, no replay. Cost: operational complexity	G4, G13
002 Polyglot storage	Mixed access patterns	PostgreSQL+pgvector, Redis, ClickHouse, Kafka	Single SQL DB: 4.2× slower inserts, no sub-ms state. Cost: four systems to run	G14
003 Delivery semantics	No loss, bounded duplicates	At-least-once + idempotent sinks	Kafka transactions: complexity, throughput cost	G13 reconciliation
005 Agent guardrails	Safe actions by an LLM agent	Policy in code outside the model, human approval	Prompt-only policy: bypassable	G10
006 Tenant isolation	No cross-tenant leakage	Forced RLS + composite FKs	Schema-per-tenant: operational cost at 100s of tenants	G1 RLS suite
008 Map technology	Real geography without keys	MapLibre GL + OpenFreeMap, GeoJSON clustering, blank-style fallback	Leaflet/raster: heavier at 800+ markers; vendor SDKs need keys	live map artefacts
009 Session lifecycle	Repeated sign-out at token expiry	Refresh-token grant, single-flight, one retry	Longer token lifetime: weaker security	auth.test.ts

All ten ADRs: `docs/adr/`.

25 Requirement traceability matrix

60 requirements: PASS 28 · PARTIAL 28 · BLOCKED 2 · FAIL 2 · NOT VERIFIED 0 (unverified items are carried as PARTIAL/BLOCKED with the gap stated).

Counts are computed from `docs/compliance/FINAL_REQUIREMENT_MATRIX.md` by the machine-checked gate `tools/reqtrace` (a PASS needs an existing evidence file; the gate checks existence, not content). The full 10-column matrix with evidence paths is in the technical artifacts.

ID	Requirement	Status	Gap / limitation
MB-01	Works on simulated data from at least 100,000 vehicles generated by our own simulator	PASS	-
MB-02	Processes events in real time and supports batch analytics over history	PASS	batch not run at billion-row scale
MB-03	Relational, NoSQL and vector storage, each justified	PASS	-
MB-04	Insights through secure APIs and a usable web interface	PASS	accessibility scan not run; real OpenFreeMap basemap not seen rendering in the sandbox browser (markers-only fallback verified)
MB-05	Containerised, tested, deployable on at least one cloud with no code change for another	PARTIAL	no cloud apply (needs the owner's cloud account); Codespaces is a LIVE DEMO ENVIRONMENT, not production cloud
TE-01	IoT ingestion: realistic generator (bursty, out-of-order, duplicate), idempotent, schema-validated, back-press	PASS	MQTT is not used (HTTPS mTLS + Kafka, ADR-004)
TE-02	Messaging: durable, partitioned streams with replay	PASS	single broker locally
TE-03	Big data pipeline: real-time detection within seconds plus batch analytics over billions of rows	PARTIAL	billion-row run not done (a ClickHouse container OOM-limited the history benchmark); latency target not met at 100K ev/s
TE-04	Persistence: polyglot with a reason for each store	PASS	object store cold tier (Parquet) not implemented
TE-05	Full stack: secure, paginated, rate-limited APIs and a clear UI	PASS	not measured under concurrent load or over a network
TE-06	ML: a model that turns data into a decision, evaluated against a baseline	PASS	labels come from the simulator's physics so absolute errors do not transfer to real fleets; no Go evaluator for the GBM, serving uses the EWMA es
TE-07	Vector layer	PASS	embedder is a hashing embedder, not a learned model
TE-08	Agentic AI: answers or acts on fleet data with guardrails and an audit trail	PASS	real LLM provider not exercised (deterministic provider; no API key)
TE-09	Log analytics: centralised logs, metrics and traces	PARTIAL	no Loki/ELK/Tempo
TE-10	DevOps: cloud-native microservices, IaC, CI/CD, cloud-agnostic deployment	PARTIAL	terraform not applied
DE-01	Relational core in 3NF with documented denormalisation	PASS	deliberate denormalisations are listed in docs/3nf.md
DE-02	Partitioning by time and vehicle; shard keys avoid hot spots	PASS	skew histogram not produced
DE-03	Data generation: 100,000+ vehicles with trips, faults, noise	PASS	-
DE-04	SQL optimisation: EXPLAIN ANALYZE before and after	PASS	-
AL-01	Graph: nearest reachable charger under range; depot clusters	PASS	-
AL-02	Dynamic programming: charging schedule under time-of-use tariffs; trip segmentation	PARTIAL	evidence is the test suite plus the write-up
AL-03	Regex and parsing: VIN validation, OBD-II codes	PASS	-
AL-04	Streaming algorithms: windows, Bloom de-dup, Count-Min top-K, geohash	PARTIAL	no general time-window aggregation operator
SD-01	CAP/PACELC choice per data kind	PASS	HA replication behaviour not exercised
SD-02	Partitioning, replication, idempotency, exactly-once vs at-least-once, back-pressure, circuit breakers, CQRS/e	PARTIAL	no circuit-breaker library (bounded retries), scale-out not run, cache-invalidation bound not tested
SD-03	Data lifecycle: hot/warm/cold with retention and cost estimate	PARTIAL	Parquet cold archive not implemented
SD-04	Microservices with clear boundaries, hexagonal internals, ADRs	PASS	-
NF-01	Sustain 100,000+ events/s	PARTIAL	below target and short; not sustained beyond 60 s
NF-02	Survive a 3x burst for 5 minutes without data loss	PARTIAL	the 5-minute duration was not run
NF-03	Ingest to dashboard under 2 s	PARTIAL	ingest-to-dashboard latency NOT VERIFIED end to end; map refresh interval is 4 s
NF-04	Critical alert under 5 s	FAIL	not met at 100K ev/s on one host; multi-host sizing not tested
NF-05	API p95 under 200 ms and p99 under 500 ms	PASS	not measured at 100K ev/s ingest
NF-06	Stateless services scale horizontally; add brokers or nodes without code change	PARTIAL	no scale-out experiment run
NF-07	No single point of failure; 99.9 % target; recovery after broker or pod kill	PARTIAL	single instances locally (SPOF by construction); availability not measured; pod kills not run
NF-08	OAuth2/OIDC + JWT, RBAC with tenant isolation, mTLS for devices, TLS 1.3, secrets in a vault	PARTIAL	Vault is dev-mode in the demo; passwords come from a generated .env, not a secret manager; AES-256 at rest only in IaC
NF-09	OWASP Top 10 / API Top 10	PARTIAL	three OWASP items have no dedicated test
NF-10	Audit logs for every data access and AI action; location masking; retention and erasure (GDPR, DPDP)	PARTIAL	retention purge for PostgreSQL audit not implemented; no legal compliance claim
TS-01	Unit tests with 80%+ coverage on core services and algorithms	PARTIAL	coverage for the later packages (rangerisk, api, copilot, batch) was not measured
TS-02	Integration tests with real broker, databases, cache (Testcontainers)	PASS	-
TS-03	Contract tests between services (Pact)	PARTIAL	Pact itself was not added
TS-04	Acceptance tests as BDD scenarios	FAIL	not implemented; the browser E2E scripts are assertion suites, not Gherkin
TS-05	Performance and load tests: throughput, p95/p99, consumer lag; one soak	PARTIAL	no soak was run; no multi-hour or 5-minute 3x burst; alert p99 < 5 s not met at 100K ev/s
TS-06	Security tests: SAST, DAST, dependency and image scans	PARTIAL	DAST is an unauthenticated baseline only (no authenticated or active scan) and ran before the CSP was tightened; not re-run
TS-07	Compliance and chaos: audit trail, erasure, kill a pod or broker and prove recovery	PARTIAL	pod kill on Kubernetes not run
DL-01	Solution Document using the provided template	PARTIAL	the template was not provided in the repository or workspace
DL-02	Repository with README, one-command local setup and a seeded 100K-vehicle dataset	PARTIAL	no run from a clean clone on a machine other than the build sandbox (the Codespaces path was not exercised)
DL-03	Architecture diagram, ER diagram (3NF), 3-5 ADRs	PARTIAL	separate runtime, trust-boundary, deployment and observability diagrams are not drawn
DL-04	Working end-to-end demo on the simulated stream	PASS	public URL depends on the owner running the Codespace; not tested from this environment
DL-05	Test evidence: coverage, load, security scans, CI link	PARTIAL	coverage incomplete (TS-01)
DL-06	DevOps pack: Dockerfiles, Helm, Terraform for one cloud, STRIDE threat model	PASS	terraform not applied
DL-07	Algorithms and SQL write-up with complexity analysis and before/after plans	PASS	-
DL-08	Demo video of 5 minutes maximum	PARTIAL	narration is synthetic speech, not a person; the OpenFreeMap basemap is not shown (markers-only fallback); recorded on the build sandbox's deploy
RL-01	Synthetic or public data only; no real personal data	PASS	-
RL-02	Open-source and AI tools declared	PASS	line-by-line human review cannot be attested by the assistant
RL-03	Final commit tagged v1.0-submission	BLOCKED	the tag was not created on GitHub by the assistant; later work is on newer commits and the tag is intentionally not moved

ID	Requirement	Status	Gap / limitation
RL-04	Originality and no affiliation with Motorq	PASS	-
LV-01	Auth session persists: no repeated sign-out; refresh before expiry; one coordinated refresh; SSE reconnect	PASS	the 330 s soak script did not complete; the 300 s production lifetime was not held for 5+ minutes
LV-02	Real geographic map with backend vehicle and charger positions, clustering, live updates, city filters, real r	PARTIAL	real OpenFreeMap basemap NOT VERIFIED (sandbox browser rejects the proxy-resigned tile-host certificate): markers-only fallback verified; stack test r
LV-03	Public HTTPS URL for judges	PARTIAL	LIVE DEMO ENVIRONMENT, not a production cloud; stops when idle; this environment did not test the public URL
LV-04	Cloud deployment (production)	BLOCKED	needs the owner's cloud account

26

Measured boundaries & remaining engineering work

These are the places where measurement stops. The architecture is designed to extend past them — partitions, stateless consumers and idempotent sinks make scale-out and replay a configuration matter — but extension was not demonstrated and is not claimed.

Boundary	What was measured	Remaining work
Critical-alert latency	p50 9.8 s / p99 17.0 s at 100K ev/s on one host (target < 5 s) — FAIL	Run the pipeline on separate hosts and re-measure; the multi-host result is unknown
Sustained throughput	98.3K ev/s for 60 s	Longer runs and ≥ 100K ev/s on multi-node hardware
3× burst	40 s inside a 100 s run, 0 lost	5-minute burst
Soak / scale-out	not run	Multi-hour soak; scale-out experiment
Dashboard < 2 s	not measured end to end	Instrument device → browser latency
Geographic basemap	markers, clusters, filters, route verified from backend data; the OpenFreeMap basemap tiles were not reachable from the build sandbox (certificate interception), so basemap rendering was not observed here	Confirm in a normal browser
BDD / Pact	none	Gherkin acceptance suite; consumer-driven contracts
DAST	unauthenticated ZAP baseline, before CSP tightening	Authenticated + active scan, re-run
Cold tier / tracing	no Parquet archive, no Loki/Tempo	Implement lifecycle job; add tracing backend
Cloud	Terraform validated, never applied; Vault dev mode	Apply in an owner-controlled account; production Vault
LLM provider	stub only	Exercise the Anthropic provider with a key
Demo video	provided as a separate MP4 (see manifest)	—

27

AI and open-source declaration

AI tools. This project was built with Claude Code (Anthropic) as an AI coding assistant: it generated code, tests, documentation and this submission package under the team's direction. The team reviewed outputs through the committed tests, scans and measurements cited above; claims were kept to what those artefacts show. The narration of the demo video is synthetic speech (Piper TTS, voice model `en_US-ryan-high`), not a recording of a person. The Copilot's default provider is a deterministic stub; no external LLM produced any answer shown in this document.

Open source. Go libraries (franz-go, pgx, go-redis, prometheus client, protobuf), Kafka, PostgreSQL + pgvector, Redis, ClickHouse, Keycloak, HashiCorp Vault, Caddy, Prometheus, Grafana, OpenTelemetry, React, Vite, MapLibre GL JS (BSD-3-Clause), scikit-learn, Playwright, Mermaid. Versions are in `go.mod`, `web/package-lock.json`, `ml/requirements.txt`; SBOM at `evidence/security/sbom-image.cdx.json`.

Data. Entirely synthetic: own simulator and seed generator (deterministic, seeded). Basemap tiles/style: OpenFreeMap (© OpenMapTiles, © OpenStreetMap contributors); no API key. Full declaration: `docs/AI_OSS_DISCLOSURE.md`.

28

Reproducibility

```
git clone https://github.com/SohamB4746Y/voltsight-connected-vehicle-intelligence.git
cd voltsight-connected-vehicle-intelligence
deploy/live/up.sh           # whole platform + 2,000-vehicle simulator; prints URL and demo logins
                             # (prerequisites: Docker with Compose v2, python3, curl)
# developer workflow: make up && make seed && make demo      (100,000-vehicle seed; console http://localhost:8081)
make test                 # unit tests (no Docker);      make test-integration # real stack
make e2e                  # live 100K-vehicle run + reconciliation (evidence under evidence/)
go run ./tools/reqtrace docs/compliance/FINAL_REQUIREMENT_MATRIX.md # traceability gate
```

Demo passwords are generated per deployment into the git-ignored `.env` and printed by `up.sh`; they are not published. The one-command path was run in the build sandbox; it has not been run from a clean clone on another machine.

29

Demo guide (5 minutes)

Time	Show	Say
0:00	Dashboard, live	100K vehicles → decisions every second; VoltSight closes the loop
0:20	Problem	Range uncertainty, charger outages, cost, latency
0:50	Architecture (Figure 1)	mTLS gateway → Kafka → worker + risk engine → stores → API → console
1:25	Dashboard cards and feed	Registered fleet, vehicles reporting live, critical/warning counts, chargers
2:05	Map: clusters, city filter	Risk concentration; basemap fallback explained
2:45	Alert evidence + route	SoC, usable range, nearest available charger, A* route
3:20	Copilot	Tool allow-list, tenant from JWT, citations, human approval, audit
3:55	Engineering depth	Prometheus, SQL plans, security, matrix
4:20	Requirement matrix	PASS / PARTIAL / FAIL with evidence

Script with voice-over text: `docs/submission/DEMO_SCRIPT.md`.

30 Conclusion

VoltSight demonstrates, on one host and on synthetic data, a complete path from vehicle telemetry to an operator decision: secure ingestion, a durable stream, a graph-based range-risk engine with a measured $O(1)$ decision, evidence-carrying alerts, a live console and map, an audited Copilot, and an ML model that beats its baselines. It is equally explicit about where measurement stops: the < 5 s critical-alert target was missed at 100K events/s on a single host, throughput peaked at 98.3K events/s for 60 s, and soak, scale-out, BDD/Pact and production-cloud deployment remain to be done. The design — partitioned log, stateless consumers, idempotent sinks, tenant isolation in the database — is what makes closing those gaps an engineering exercise rather than a redesign.